

Muestreo

```
library(ggplot2)
```

Warning: package 'ggplot2' was built under R version 4.4.3

```
data(diamonds)
diamonds
```

```
# A tibble: 53,940 x 10
  carat cut      color clarity depth table price      x      y      z
  <dbl> <ord>    <ord> <ord>    <dbl> <dbl> <int> <dbl> <dbl> <dbl>
1  0.23 Ideal      E      SI2     61.5    55   326  3.95  3.98  2.43
2  0.21 Premium    E      SI1     59.8    61   326  3.89  3.84  2.31
3  0.23 Good       E      VS1     56.9    65   327  4.05  4.07  2.31
4  0.29 Premium    I      VS2     62.4    58   334  4.2   4.23  2.63
5  0.31 Good       J      SI2     63.3    58   335  4.34  4.35  2.75
6  0.24 Very Good  J      VVS2     62.8    57   336  3.94  3.96  2.48
7  0.24 Very Good  I      VVS1     62.3    57   336  3.95  3.98  2.47
8  0.26 Very Good  H      SI1     61.9    55   337  4.07  4.11  2.53
9  0.22 Fair       E      VS2     65.1    61   337  3.87  3.78  2.49
10 0.23 Very Good  H      VS1     59.4    61   338  4     4.05  2.39
# i 53,930 more rows
```

Contiene datos sobre clientes de un banco (edad, trabajo, saldo, si aceptaron un préstamo).

```
library(tidyverse)
```

Warning: package 'tidyverse' was built under R version 4.4.3

Warning: package 'tibble' was built under R version 4.4.3

Warning: package 'tidyr' was built under R version 4.4.3

Warning: package 'readr' was built under R version 4.4.3

Warning: package 'purrr' was built under R version 4.4.3

Warning: package 'dplyr' was built under R version 4.4.3

Warning: package 'stringr' was built under R version 4.4.3

Warning: package 'forcats' was built under R version 4.4.3

Warning: package 'lubridate' was built under R version 4.4.3

-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --

v dplyr 1.1.4 v readr 2.1.5

v forcats 1.0.0 v stringr 1.5.1

v lubridate 1.9.4 v tibble 3.2.1

v purrr 1.0.4 v tidyr 1.3.1

-- Conflicts ----- tidyverse_conflicts() --

x dplyr::filter() masks stats::filter()

x dplyr::lag() masks stats::lag()

i Use the conflicted package (<<http://conflicted.r-lib.org/>>) to force all conflicts to become

```
# Carga de datos (puedes descargarlo o leerlo directamente si tienes el link)
url <- "https://archive.ics.uci.edu/ml/machine-learning-databases/00222/bank.zip"
temp <- tempfile()
download.file(url, temp)
bank_data <- read.table(unz(temp, "bank.csv"), sep = ";", header = TRUE)
unlink(temp)

# Ver la estructura
str(bank_data)
```

'data.frame': 4521 obs. of 17 variables:

\$ age : int 30 33 35 30 59 35 36 39 41 43 ...

\$ job : chr "unemployed" "services" "management" "management" ...

\$ marital : chr "married" "married" "single" "married" ...

\$ education: chr "primary" "secondary" "tertiary" "tertiary" ...

\$ default : chr "no" "no" "no" "no" ...

```

$ balance : int 1787 4789 1350 1476 0 747 307 147 221 -88 ...
$ housing : chr "no" "yes" "yes" "yes" ...
$ loan : chr "no" "yes" "no" "yes" ...
$ contact : chr "cellular" "cellular" "cellular" "unknown" ...
$ day : int 19 11 16 3 5 23 14 6 14 17 ...
$ month : chr "oct" "may" "apr" "jun" ...
$ duration : int 79 220 185 199 226 141 341 151 57 313 ...
$ campaign : int 1 1 1 4 1 2 1 2 2 1 ...
$ pdays : int -1 339 330 -1 -1 176 330 -1 -1 147 ...
$ previous : int 0 4 1 0 0 3 2 0 0 2 ...
$ poutcome : chr "unknown" "failure" "failure" "unknown" ...
$ y : chr "no" "no" "no" "no" ...

```

```
#bank_data
```

```
sum(is.na(bank_data))
```

```
[1] 0
```

```
# Resumen estadístico
```

```
summary(bank_data$balance)
```

```

      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
-3313      69      444    1423    1480    71188

```

```
# Visualización de la distribución
```

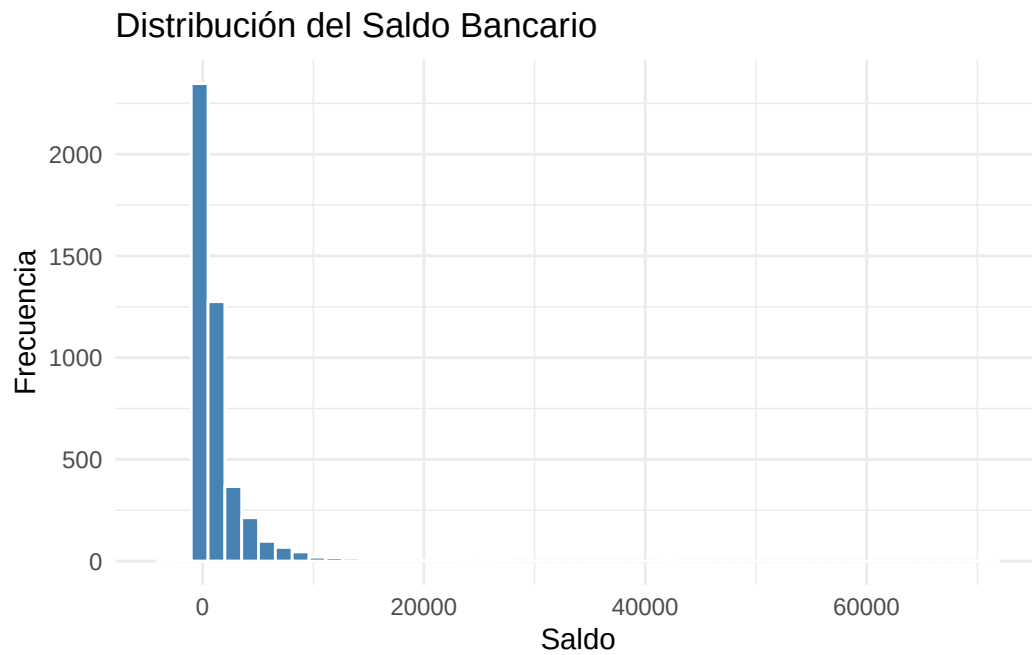
```
library(ggplot2)
```

```
ggplot(bank_data, aes(x = balance)) +
```

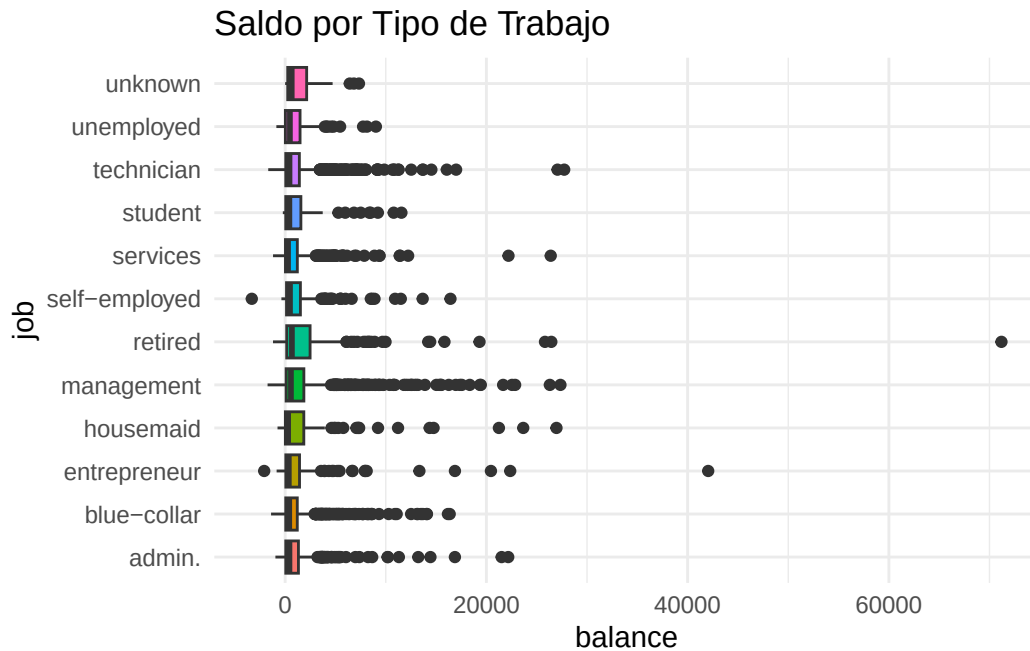
```
  geom_histogram(bins = 50, fill = "steelblue", color = "white") +
```

```
  theme_minimal() +
```

```
  labs(title = "Distribución del Saldo Bancario", x = "Saldo", y = "Frecuencia")
```



```
ggplot(bank_data, aes(x = job, y = balance, fill = job)) +  
  geom_boxplot() +  
  coord_flip() + # Para leer mejor los nombres de los trabajos  
  theme_minimal() +  
  theme(legend.position = "none") +  
  labs(title = "Saldo por Tipo de Trabajo")
```



```
# 1. Real
media_real <- mean(bank_data$balance)
```

```
# 2. MAS (Simple)
set.seed(123) # Para que tus resultados sean reproducibles
muestra_simple <- bank_data %>% sample_frac(0.05)
media_mas <- mean(muestra_simple$balance)
```

```
# 3. Estratificado (por job)
muestra_estrat <- bank_data %>%
  group_by(job) %>%
  sample_frac(0.05) %>%
  ungroup()
media_estrat <- mean(muestra_estrat$balance)
```

```
# Definimos la probabilidad de inclusión (ej. 5%)
prob_inclusion <- 0.05

set.seed(123)
# Cada fila tiene su propio "lanzamiento de moneda"
muestra_bernoulli <- bank_data %>%
  filter(rbinom(n(), 1, prob_inclusion) == 1)
```

```
# ¿Cuántos clientes terminaron en la muestra?
nrow(muestra_bernoulli)
```

```
[1] 216
```

```
# Estimador del Total de saldos (Horvitz-Thompson)
total_est_ht <- sum(muestra_bernoulli$balance / probab_inclusion)

# Estimador de la Media (Hajek)
media_est_bernoulli <- sum(muestra_bernoulli$balance / probab_inclusion) /
  sum(1 / probab_inclusion)

# Comparar con la real
media_real <- mean(bank_data$balance)
print(paste("Real:", media_real, "| Bernoulli:", media_est_bernoulli))
```

```
[1] "Real: 1422.65781906658 | Bernoulli: 286870"
```

```
# Comparación rápida
data.frame(Metodo = c("Real", "Simple", "Estratificado"),
  Media = c(media_real, media_mas, media_estrat))
```

	Metodo	Media
1	Real	1422.658
2	Simple	1541.482
3	Estratificado	1472.569

```
set.seed(123)
simulaciones <- 1000
resultados <- data.frame(metodo = character(), media = numeric())

for(i in 1:simulaciones) {
  # MAS (Tamaño fijo)
  m_mas <- bank_data %>% sample_n(226)
  resultados <- rbind(resultados, data.frame(metodo = "MAS", media = mean(m_mas$balance)))

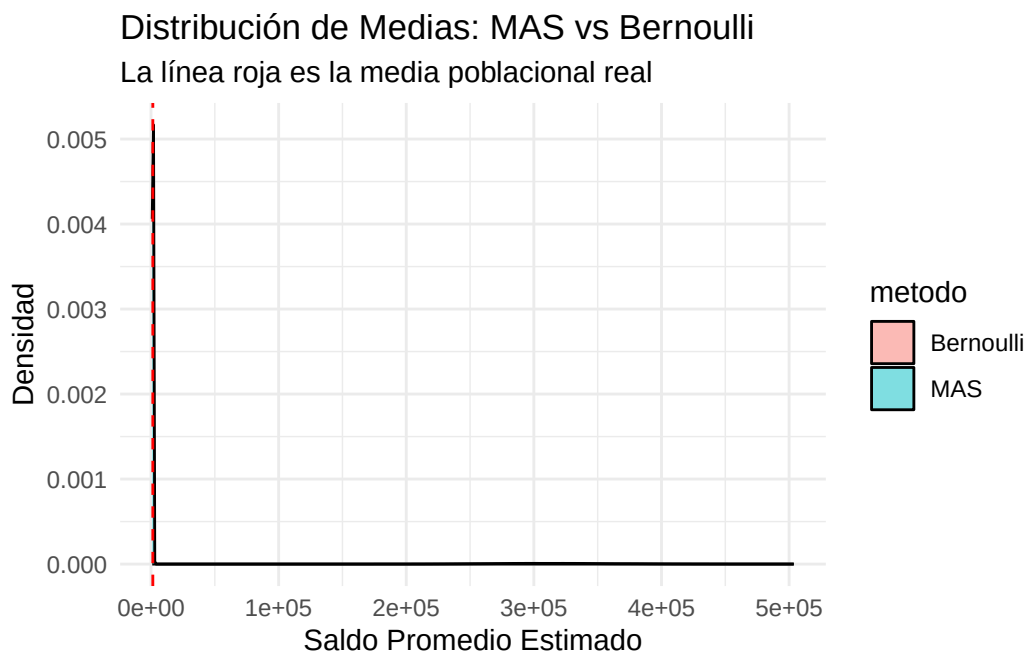
  # Bernoulli (Tamaño aleatorio)
  m_bern <- bank_data %>% filter(rbinom(n(), 1, 0.05) == 1)
  # Usamos la media de Hajek para mayor precisión
```

```

media_h <- sum(m_bern$balance / 0.05) / sum(1 / 0.05)
resultados <- rbind(resultados, data.frame(metodo = "Bernoulli", media = media_h))
}

# Visualización
ggplot(resultados, aes(x = media, fill = metodo)) +
  geom_density(alpha = 0.5) +
  geom_vline(xintercept = media_real, linetype = "dashed", color = "red") +
  theme_minimal() +
  labs(title = "Distribución de Medias: MAS vs Bernoulli",
       subtitle = "La línea roja es la media poblacional real",
       x = "Saldo Promedio Estimado", y = "Densidad")

```



```

library(tidyverse)

# Asumiendo que ya tienes el dataframe 'resultados' de la simulación anterior
# con las columnas 'metodo' y 'media'

media_real <- mean(bank_data$balance)

metricas <- resultados %>%
  group_by(metodo) %>%

```

```

summarise(
  Media_Estimadores = mean(media),
  Sesgo = mean(media) - media_real,
  Varianza = var(media),
  MSE = mean((media - media_real)^2),
  RMSE = sqrt(MSE),
  CV = (sd(media) / mean(media)) * 100
)

print(metricas)

```

```

# A tibble: 2 x 7
  metodo      Media_Estimadores      Sesgo      Varianza      MSE      RMSE      CV
  <chr>          <dbl>      <dbl>      <dbl>      <dbl>      <dbl> <dbl>
1 Bernoulli    317694.  316271.  2340660053. 102365794619. 319947.  15.2
2 MAS          1426.    3.77    35167.    35146.    187.   13.1

```

```

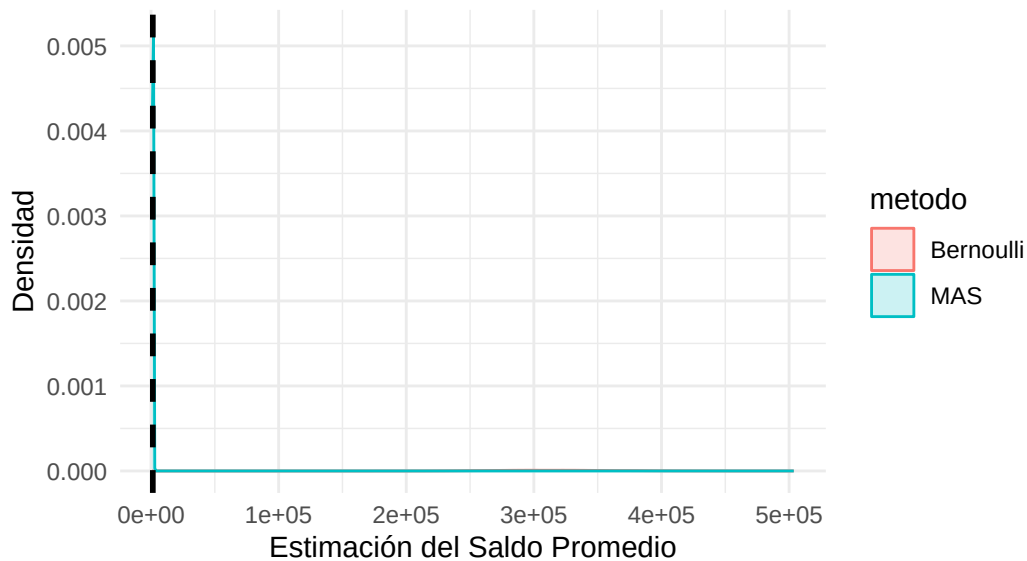
ggplot(resultados, aes(x = media, color = metodo, fill = metodo)) +
  geom_density(alpha = 0.2) +
  geom_vline(xintercept = media_real, linetype = "dashed", size = 1) +
  labs(
    title = "Distribución de los Estimadores vs. Valor Real",
    subtitle = "La cercanía al centro es el Sesgo; la anchura es la Varianza",
    x = "Estimación del Saldo Promedio",
    y = "Densidad"
  ) +
  theme_minimal()

```

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
 i Please use `linewidth` instead.

Distribución de los Estimadores vs. Valor Real

La cercanía al centro es el Sesgo; la anchura es la Varianza



```
library(tidyverse)

# 1. Definimos probabilidades desiguales
# Clientes con hipoteca: 10% de prob. | Sin hipoteca: 5% de prob.
bank_data <- bank_data %>%
  mutate(pi_i = ifelse(housing == "yes", 0.10, 0.05))

# 2. Extraemos la muestra basada en esas probabilidades (Bernoulli Desigual)
set.seed(456)
muestra_ht <- bank_data %>%
  filter(runif(n()) <= pi_i)

# 3. Aplicamos el Estimador de Horvitz-Thompson para el TOTAL del Balance
total_poblacional_real <- sum(bank_data$balance)

total_est_ht <- sum(muestra_ht$balance / muestra_ht$pi_i)

# 4. Estimador de la MEDIA (Hajek)
# El estimador HT para la media usa el tamaño poblacional N
N <- nrow(bank_data)
media_est_ht <- total_est_ht / N

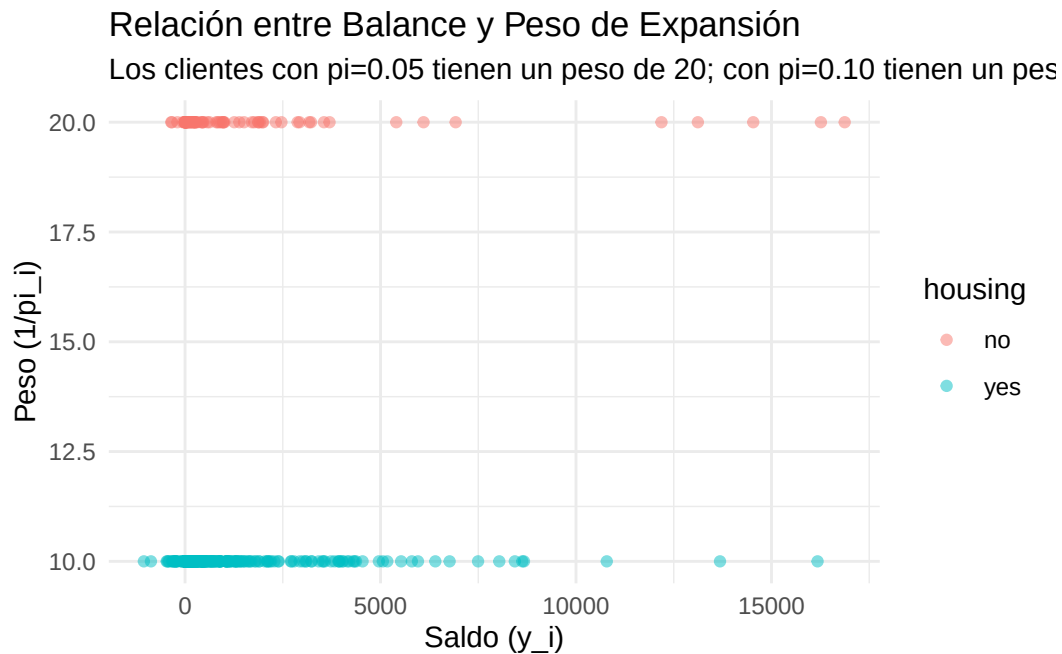
# Comparación
```

```
data.frame(
  Concepto = c("Total", "Media"),
  Real = c(total_poblacional_real, mean(bank_data$balance)),
  Estimado_HT = c(total_est_ht, media_est_ht)
)
```

	Concepto	Real	Estimado_HT
1	Total	6431836.000	6100930.000
2	Media	1422.658	1349.465

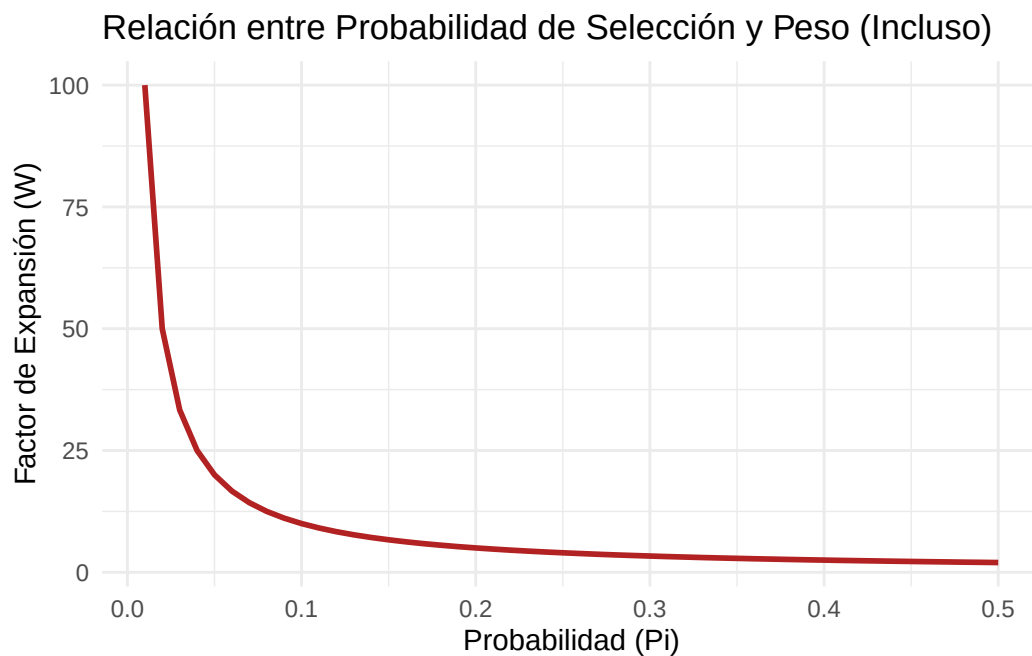
```
muestra_ht <- muestra_ht %>%
  mutate(peso = 1 / pi_i)

ggplot(muestra_ht, aes(x = balance, y = peso, color = housing)) +
  geom_point(alpha = 0.5) +
  labs(title = "Relación entre Balance y Peso de Expansión",
       subtitle = "Los clientes con pi=0.05 tienen un peso de 20; con pi=0.10 tienen un peso",
       x = "Saldo (y_i)", y = "Peso (1/pi_i)") +
  theme_minimal()
```



```
# Probabilidad de seleccion
# Visualizando la relación inversa
ejemplo_pesos <- data.frame(
  Probabilidad = seq(0.01, 0.5, by = 0.01)
) %>%
  mutate(Peso_Expansion = 1 / Probabilidad)

ggplot(ejemplo_pesos, aes(x = Probabilidad, y = Peso_Expansion)) +
  geom_line(color = "firebrick", size = 1) +
  labs(title = "Relación entre Probabilidad de Selección y Peso (Incluso)",
       x = "Probabilidad (Pi)", y = "Factor de Expansión (W)") +
  theme_minimal()
```



SIR y Con reemplazo (Simple)

```
n_muestra <- 500

# MAS Sin Reemplazo (Por defecto en R)
set.seed(123)
muestra_sin <- bank_data[sample(nrow(bank_data), n_muestra, replace = FALSE), ]

# MAS Con Reemplazo
set.seed(123)
```

```
muestra_con <- bank_data[sample(nrow(bank_data), n_muestra, replace = TRUE), ]

# ¿Hubo repetidos en la muestra con reemplazo?
anyDuplicated(muestra_con) # Si devuelve > 0, hay filas repetidas
```

```
[1] 59
```

```
sim_varianza <- replicate(1000, {
  m_sin <- bank_data$balance[sample(nrow(bank_data), 200, replace = FALSE)]
  m_con <- bank_data$balance[sample(nrow(bank_data), 200, replace = TRUE)]
  c(media_sin = mean(m_sin), media_con = mean(m_con))
})

# Convertir a data frame y comparar desviaciones estándar
sim_res <- as.data.frame(t(sim_varianza))
sd(sim_res$media_sin) # Debería ser ligeramente menor
```

```
[1] 207.0137
```

```
sd(sim_res$media_con)
```

```
[1] 209.4216
```

estimador pwr

```
library(tidyverse)

# 1. Definir probabilidades de selección (p_i)
# Deben sumar 1 en toda la población
bank_data <- bank_data %>%
  mutate(prioridad = case_when(
    education == "tertiary" ~ 3,
    education == "secondary" ~ 2,
    TRUE ~ 1
  )) %>%
  mutate(p_i = prioridad / sum(prioridad))

# 2. Realizar el muestreo CON reemplazo (n = 500)
n_muestras <- 500
```

```

set.seed(789)
indices_pwr <- sample(1:nrow(bank_data), size = n_muestras, replace = TRUE, prob = bank_data$balance)
muestra_pwr <- bank_data[indices_pwr, ]

# 3. Calcular el Estimador pwr para el TOTAL
# Aplicamos la fórmula:  $(1/n) * \sum(y_i / p_i)$ 
total_pwr <- (1 / n_muestras) * sum(muestra_pwr$balance / muestra_pwr$p_i)

# 4. Calcular la MEDIA estimada
media_pwr <- total_pwr / nrow(bank_data)

# Comparación
media_real <- mean(bank_data$balance)
print(paste("Media Real:", round(media_real, 2)))

```

```
[1] "Media Real: 1422.66"
```

```
print(paste("Media Estimador pwr:", round(media_pwr, 2)))
```

```
[1] "Media Estimador pwr: 1218.67"
```

```

# Cálculo de la varianza del total en R
valores_expandidos <- muestra_pwr$balance / muestra_pwr$p_i
var_total_pwr <- (1 / (n_muestras * (n_muestras - 1))) * sum((valores_expandidos - total_pwr)^2)

# Error estándar de la media
se_media_pwr <- sqrt(var_total_pwr) / nrow(bank_data)

```

Efecto Diseño (deff)

```

# Instalación y carga
if(!require(survey)) install.packages("survey")

```

Cargando paquete requerido: survey

Warning: package 'survey' was built under R version 4.4.3

Cargando paquete requerido: grid

Cargando paquete requerido: Matrix

Adjuntando el paquete: 'Matrix'

The following objects are masked from 'package:tidyr':

expand, pack, unpack

Cargando paquete requerido: survival

Adjuntando el paquete: 'survey'

The following object is masked from 'package:graphics':

dotchart

```
library(survey)

# 1. Definir el diseño de muestreo (ej. Estratificado por Educación)
# Usamos 'fpc' para indicar que es sin reemplazo
diseno_estrat <- svydesign(
  id = ~1,
  strata = ~education,
  data = bank_data,
  fpc = rep(nrow(bank_data), nrow(bank_data))
)

# 2. Calcular la media y pedir el DEFF
resultado <- svymean(~balance, diseno_estrat, deff = TRUE)

# Ver el resultado
print(resultado)
```

	mean	SE	DEff
balance	1521.257	66.297	2.6068

```
deff_valor <- deff(resultado)
print(paste("El Efecto de Diseño es:", round(deff_valor, 4)))
```

```
[1] "El Efecto de Diseño es: 2.6068"
```

Estimacion por dominios

```
library(survey)

# 1. Definimos el diseño completo primero (importante)
diseno_completo <- svydesign(
  id = ~1,
  data = bank_data,
  fpc = rep(nrow(bank_data), nrow(bank_data))
)

# 2. Creamos el dominio (subpoblación de interés)
# Ejemplo: Queremos estimar el balance solo para personas con educación 'tertiary'
diseno_dominio <- subset(diseno_completo, education == "tertiary")

# 3. Estimación
estimacion_dom <- svymean(~balance, diseno_dominio)
print(estimacion_dom)
```

```
      mean SE
balance 1775.4 0
```

```
# 4. Comparación: ¿Cómo cambia el error si el dominio es pequeño?
# Intentemos con un dominio más específico (ej. 'entrepreneur' con deuda)
diseno_pequeno <- subset(diseno_completo, job == "entrepreneur" & loan == "yes")
svymean(~balance, diseno_pequeno)
```

```
      mean SE
balance 1143.3 0
```

```
# Estimación del total en el dominio
svytotal(~balance, diseno_dominio)
```

```
      total SE
balance 2396822 0
```

Diseño Sistemático

```
# 1. Parámetros
N <- nrow(bank_data)
n <- 500
k <- floor(N / n)

# 2. Arranque aleatorio
set.seed(123)
arranque <- sample(1:k, 1)

# 3. Selección de índices
indices_sist <- seq(from = arranque, to = N, by = k)

# 4. Crear la muestra
muestra_sistemica <- bank_data[indices_sist, ]

# Verificación
nrow(muestra_sistemica)
```

```
[1] 503
```

```
library(survey)

# Definimos el diseño sistemático en 'survey'
# Se suele aproximar como MAS si no hay periodicidad
diseno_sist <- svydesign(
  id = ~1,
  data = muestra_sistemica,
  fpc = rep(N, nrow(muestra_sistemica))
)

svymean(~balance, diseno_sist)
```

```
      mean      SE
balance 1263.4 111.69
```

Diseño Poisson


```

library(tidyverse)

# 1. Definir probabilidades pi_i individuales
# Usamos el balance absoluto para evitar problemas con saldos negativos
# y escalamos para que la suma de pi_i sea igual al tamaño de muestra deseado (n=500)
n_deseado <- 500
bank_data <- bank_data %>%
  mutate(balance_adj = abs(balance) + 1) %>% # Evitar ceros
  mutate(pi_i = n_deseado * (balance_adj / sum(balance_adj))) %>%
  mutate(pi_i = pmin(pi_i, 1)) # Asegurar que ninguna prob sea > 1

# 2. Ejecutar el proceso de Poisson
set.seed(321)
muestra_poisson <- bank_data %>%
  filter(runif(n()) <= pi_i)

# El tamaño de muestra será cercano a 500, pero no exacto
nrow(muestra_poisson)

```

```
[1] 479
```

```

# Estimación del Total Poblacional
total_est_poisson <- sum(muestra_poisson$balance / muestra_poisson$pi_i)

# Estimación de la Media (Hajek) - Más estable para Poisson
media_est_poisson <- sum(muestra_poisson$balance / muestra_poisson$pi_i) /
  sum(1 / muestra_poisson$pi_i)

# Comparación con la realidad
media_real <- mean(bank_data$balance)
print(paste("Real:", round(media_real, 2), "| Poisson:", round(media_est_poisson, 2)))

```

```
[1] "Real: 1422.66 | Poisson: 1723.17"
```

```

# Cálculo de la varianza estimada del total
var_total_poisson <- sum((1 - muestra_poisson$pi_i) * (muestra_poisson$balance / muestra_poisson$pi_i))
var_total_poisson

```

```
[1] 50494500404
```

```
# Error estándar de la media
se_media_poisson <- sqrt(var_total_poisson) / nrow(bank_data)
```

Diseño pps y ps

```
if(!require(sampling)) install.packages("sampling")
```

Cargando paquete requerido: sampling

Warning: package 'sampling' was built under R version 4.4.3

Adjuntando el paquete: 'sampling'

The following objects are masked from 'package:survival':

cluster, strata

```
library(sampling)
library(tidyverse)

# --- Preparación de la variable de tamaño (Size) ---
# Usamos el balance absoluto para que siempre sea positivo
bank_data <- bank_data %>%
  mutate(size_var = abs(balance) + 1)

n_muestra <- 500

# 1. DISEÑO PPS (Con reemplazo)
set.seed(123)
# Calculamos p_i (probabilidades de selección en cada sorteo, suman 1)
p_i <- bank_data$size_var / sum(bank_data$size_var)
indices_pps <- sample(1:nrow(bank_data), size = n_muestra, replace = TRUE, prob = p_i)
muestra_pps <- bank_data[indices_pps, ]

# Estimador pwr (Hansen-Hurwitz)
total_pps <- (1/n_muestra) * sum(muestra_pps$balance / p_i[indices_pps])

# 2. DISEÑO pips (Sin reemplazo - Algoritmo Sistemático)
```

```

# Calculamos pi_i (probabilidades de inclusión, suman n)
pi_i <- inclusionprobabilities(bank_data$size_var, n_muestra)

set.seed(123)
# Usamos el método sistemático para pips
indicador_pips <- UPsystematic(pi_i)
muestra_pips <- bank_data[indicador_pips == 1, ]

# Estimador Horvitz-Thompson
total_pips <- sum(muestra_pips$balance / pi_i[indicador_pips == 1])

# --- Resultados ---
media_real <- mean(bank_data$balance)
data.frame(
  Metodo = c("Real", "PPS (H-H)", "piPS (H-T)"),
  Media_Estimada = c(media_real, total_pips/nrow(bank_data), total_pips/nrow(bank_data))
)

```

	Metodo	Media_Estimada
1	Real	1422.658
2	PPS (H-H)	1422.770
3	piPS (H-T)	1431.919

Diseño Estratificado

```

library(tidyverse)
library(sampling)

# 1. Preparar los datos (ordenar por estrato es buena práctica)
bank_data <- bank_data %>% arrange(job)
N <- nrow(bank_data)
n_total <- 500

# 2. Calcular tamaños de estrato en la población (Nh)
estratos_info <- bank_data %>%
  group_by(job) %>%
  summarise(Nh = n(), Sh = sd(balance)) %>%
  mutate(wh = Nh / N) # Pesos poblacionales

# 3. Afijación Proporcional (nh)
estratos_info <- estratos_info %>%

```

```

mutate(n_prop = round(n_total * wh))

# 4. Extraer la muestra estratificada (Proporcional)
set.seed(123)
muestra_estrat <- strata(bank_data,
                        stratanames = "job",
                        size = estratos_info$n_prop,
                        method = "srswor") # Sin reemplazo dentro del estrato

# 5. Recuperar los datos de la muestra
datos_muestra_estrat <- getdata(bank_data, muestra_estrat)

```

```

# Cálculo manual del estimador estratificado
estimacion_estrat <- datos_muestra_estrat %>%
  group_by(job) %>%
  summarise(media_h = mean(balance)) %>%
  left_join(estratos_info, by = "job") %>%
  summarise(media_global_st = sum(media_h * wh))

media_real <- mean(bank_data$balance)
print(paste("Media Real:", round(media_real, 2)))

```

```
[1] "Media Real: 1422.66"
```

```
print(paste("Media Estratificada:", round(estimacion_estrat$media_global_st, 2)))
```

```
[1] "Media Estratificada: 1658.96"
```

```

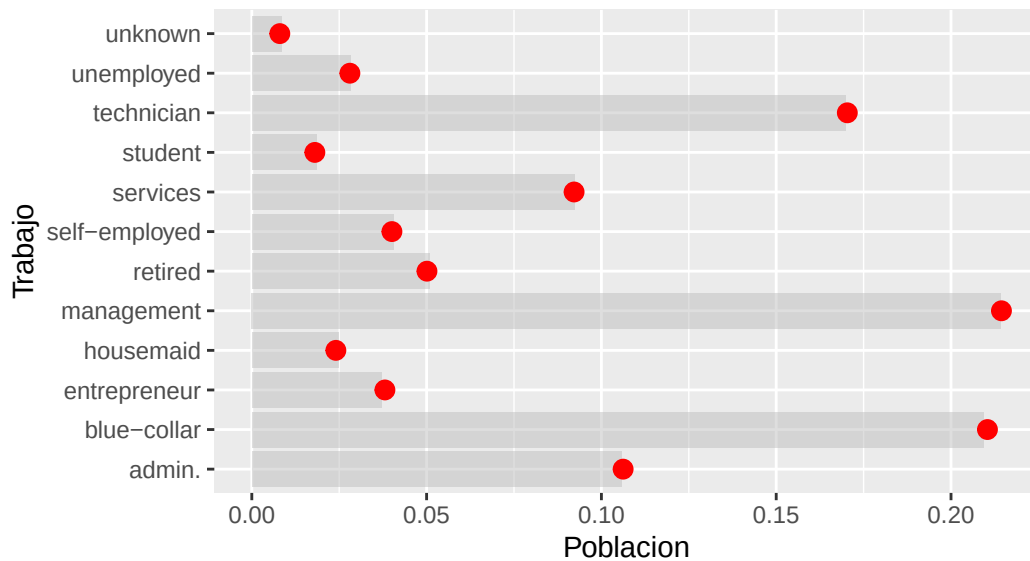
# Comparar proporciones
comp_proporciones <- data.frame(
  Trabajo = estratos_info$job,
  Poblacion = estratos_info$wh,
  Muestra = as.vector(table(datos_muestra_estrat$job) / nrow(datos_muestra_estrat))
)

ggplot(comp_proporciones, aes(x = Trabajo)) +
  geom_bar(aes(y = Poblacion), stat = "identity", fill = "gray", alpha = 0.5) +
  geom_point(aes(y = Muestra), color = "red", size = 3) +
  coord_flip() +
  labs(title = "Representatividad: Población (barras) vs Muestra (puntos)",
       subtitle = "La afijación proporcional clava las proporciones reales")

```

Representatividad: Población (barras) vs Muestra (puntos)

La afijación proporcional clava las proporciones reales



Muestreo por Conglomerados

```
library(survey)
library(tidyverse)

# 1. Simulamos 20 "sucursales" o clusters (ID_sucursal)
set.seed(456)
bank_data <- bank_data %>%
  mutate(cluster_id = sample(1:20, nrow(.), replace = TRUE))

# 2. Selección de Clusters: Vamos a elegir 4 sucursales al azar
sucursales_elegidas <- sample(1:20, size = 4)

# 3. Nuestra muestra son TODOS los clientes de esas 4 sucursales
muestra_conglomerados <- bank_data %>%
  filter(cluster_id %in% sucursales_elegidas)

# Tamaño de la muestra (es aleatorio dependiendo de cuánta gente haya en cada sucursal)
nrow(muestra_conglomerados)
```

[1] 923

```
# 4. Definir el diseño de conglomerados
# id = ~cluster_id indica que la unidad de muestreo es el cluster
diseno_clusters <- svydesign(
  id = ~cluster_id,
  data = muestra_conglomerados,
  fpc = rep(20, nrow(muestra_conglomerados)) # 20 es el total de clusters en la población
)

# 5. Estimar la media
media_clusters <- svymean(~balance, diseno_clusters, deff = TRUE)
print(media_clusters)
```

	mean	SE	DEff
balance	1498.564	86.594	0.649

Coefficiente de heterogeneidad intra-cluster

```
# Usando la librería 'ICC' o calculándolo a través de un modelo lineal
if(!require(ICC)) install.packages("ICC")
```

Cargando paquete requerido: ICC

```
library(ICC)

# Calculamos el ICC (rho) para la variable 'balance' según los 'cluster_id'
# (Usando los clusters que simulamos en el paso anterior)
resultado_icc <- ICCbare(x = factor(cluster_id), y = balance, data = bank_data)

print(paste("El Coeficiente de Correlación Intraclass (rho) es:", round(resultado_icc, 4)))
```

```
[1] "El Coeficiente de Correlación Intraclass (rho) es: -0.0012"
```

Muestreo en dos etapas:

```
library(sampling)
library(tidyverse)

library(tidyverse)
library(survey)
```

```

# 1. Definir los parámetros
n_upm <- 5 # Número de sucursales a elegir (Etapa 1)
n_usm <- 10 # Número de clientes por sucursal (Etapa 2)

# 2. ETAPA 1: Seleccionar las sucursales (UPM)
set.seed(123)
sucursales_poblacion <- unique(bank_data$cluster_id)
sucursales_elegidas <- sample(sucursales_poblacion, size = n_upm)

# 3. ETAPA 2: Seleccionar clientes dentro de esas sucursales (USM)
muestra_2etapas <- bank_data %>%
  filter(cluster_id %in% sucursales_elegidas) %>%
  group_by(cluster_id) %>%
  sample_n(size = n_usm) %>% # Elegimos 10 de cada una
  ungroup()

# 4. Verificar la muestra
print(table(muestra_2etapas$cluster_id))

```

```

3  4  7  8 10
10 10 10 10 10

```

```

library(tidyverse)
library(survey)

# 1. Parámetros del diseño
n_upm <- 5 # Cuántas sucursales elegimos
n_usm <- 10 # Cuántos clientes por sucursal
N_total_upm <- 20 # Total de sucursales en la población

# 2. Selección de la muestra
set.seed(123)
sucursales_elegidas <- sample(unique(bank_data$cluster_id), size = n_upm)

muestra_2etapas <- bank_data %>%
  filter(cluster_id %in% sucursales_elegidas) %>%
  group_by(cluster_id) %>%
  mutate(Mi = n()) %>% # Clientes totales en ESTA sucursal
  sample_n(size = n_usm) %>% # Seleccionamos 10
  ungroup()

```

```
# 3. Cálculo manual de Probabilidades y Pesos (Aquí corregimos el error)
muestra_2etapas <- muestra_2etapas %>%
  mutate(
    prob_etapa1 = n_upm / N_total_upm, # P(seleccionar sucursal)
    prob_etapa2 = n_usm / Mi,          # P(seleccionar cliente | sucursal)
    pi_total = prob_etapa1 * prob_etapa2, # Probabilidad de inclusión global
    peso_w = 1 / pi_total                # Factor de expansión (HT)
  )

# 4. Verificación: La suma de pesos debe ser cercana al N poblacional
cat("Suma de pesos (debe ser aprox", nrow(bank_data), "):", sum(muestra_2etapas$peso_w), "\n")
```

Suma de pesos (debe ser aprox 4521): 4492

```
# 5. Diseño y Estimación
diseno_final <- svydesign(
  id = ~cluster_id,
  weights = ~peso_w,
  data = muestra_2etapas
)

resultado <- svymean(~balance, diseno_final)
print(resultado)
```

	mean	SE
balance	1522.5	244.53

Supuestos de Invarianza y de Independencia

Independencia:

```
# Si rho es cercano a 0, hay "cuasi-independencia"
# Si rho es alto, confirmamos que el diseño de clusters es necesario
library(ICC)
ICCbare(x = factor(cluster_id), y = balance, data = muestra_2etapas)
```

[1] -0.01985277

Invarianza


```

# Comparamos la media del balance en dos dominios (con y sin préstamo personal)
# Si hay invarianza, el comportamiento del diseño debe ser similar en ambos

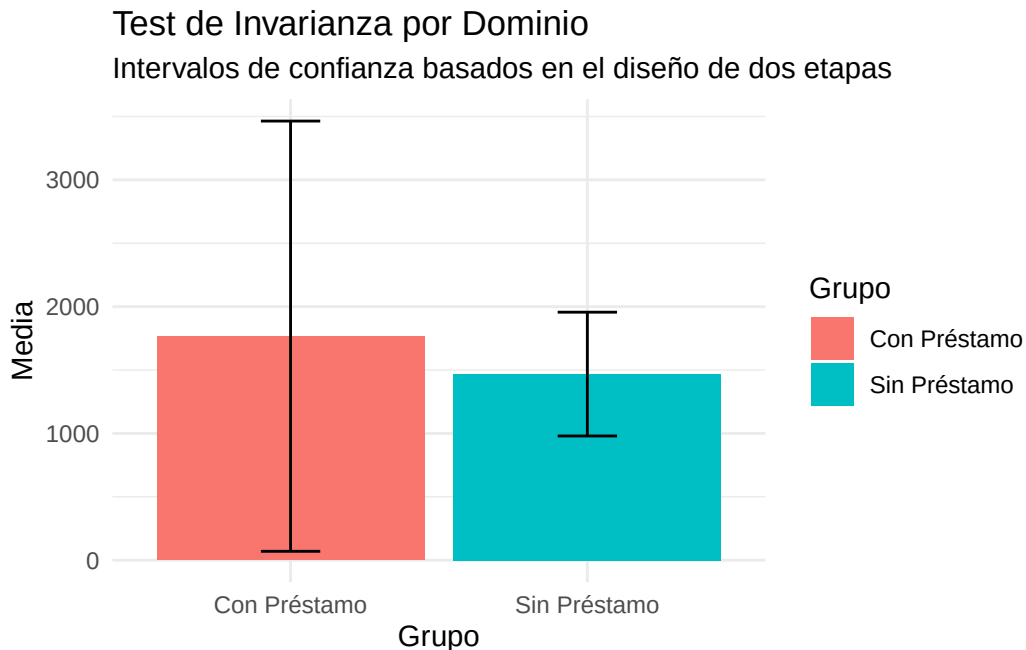
dominio_prestamo <- subset(diseno_final, loan == "yes")
dominio_no_prestamo <- subset(diseno_final, loan == "no")

mean_yes <- svymean(~balance, dominio_prestamo)
mean_no <- svymean(~balance, dominio_no_prestamo)

# Visualización de Invarianza de la Media
res_inv <- data.frame(
  Grupo = c("Con Préstamo", "Sin Préstamo"),
  Media = c(coef(mean_yes), coef(mean_no)),
  SE = c(SE(mean_yes), SE(mean_no))
)

ggplot(res_inv, aes(x = Grupo, y = Media, fill = Grupo)) +
  geom_bar(stat = "identity") +
  geom_errorbar(aes(ymin = Media - 1.96*SE, ymax = Media + 1.96*SE), width = 0.2) +
  theme_minimal() +
  labs(title = "Test de Invarianza por Dominio",
       subtitle = "Intervalos de confianza basados en el diseño de dos etapas")

```



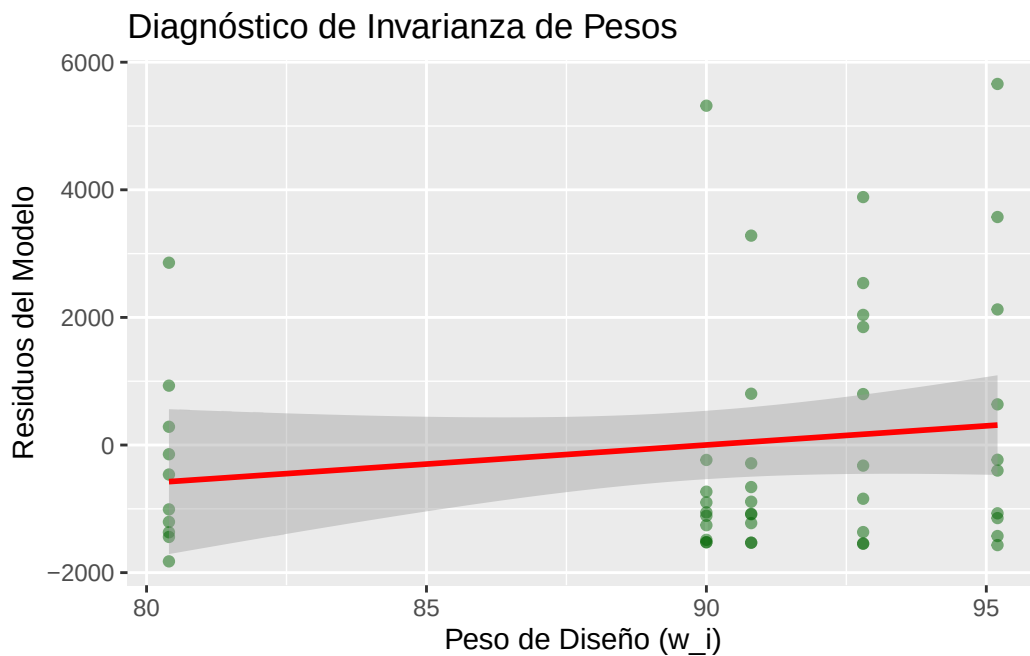
```
# Calculamos residuos del diseño
muestra_2etapas$residuos <- residuals(svyglm(balance ~ 1, design = diseno_final))

# Correlación entre Peso y Residuo
# Un valor cercano a 0 indica que el peso es invariante respecto al error
cor_test <- cor.test(muestra_2etapas$peso_w, muestra_2etapas$residuos)
print(paste("Correlación Peso-Residuo:", round(cor_test$estimate, 4)))
```

```
[1] "Correlación Peso-Residuo: 0.1617"
```

```
ggplot(muestra_2etapas, aes(x = peso_w, y = residuos)) +
  geom_point(alpha = 0.5, color = "darkgreen") +
  geom_smooth(method = "lm", color = "red") +
  labs(title = "Diagnóstico de Invarianza de Pesos",
       x = "Peso de Diseño (w_i)", y = "Residuos del Modelo")
```

`geom\_smooth()` using formula = 'y ~ x'



```
# Calculamos la varianza por cada sucursal (cluster) seleccionada
varianza_clusters <- muestra_2etapas %>%
```

```

group_by(cluster_id) %>%
  summarise(var_balance = var(balance), n = n())

# Test de Levene para homogeneidad de varianza entre clusters
# Si p > 0.05, aceptamos el supuesto de invarianza de la varianza
if(!require(car)) install.packages("car")

```

Cargando paquete requerido: car

Warning: package 'car' was built under R version 4.4.3

Cargando paquete requerido: carData

Warning: package 'carData' was built under R version 4.4.3

Adjuntando el paquete: 'car'

The following object is masked from 'package:dplyr':

recode

The following object is masked from 'package:purrr':

some

```

car::leveneTest(balance ~ factor(cluster_id), data = muestra_2etapas)

```

Levene's Test for Homogeneity of Variance (center = median)

	Df	F value	Pr(>F)
group	4	0.7374	0.5714
	45		

```

library(survey)
library(tidyverse)

```

```

# 1. Aseguramos que el diseño esté bien declarado
# (Usando los pesos calculados en el paso anterior)

```

```

diseño_final <- svydesign(
  id = ~cluster_id,
  weights = ~peso_w,
  data = muestra_2etapas
)

# 2. Calculamos la media y guardamos el objeto res_media
res_media <- svymean(~balance, diseño_final)

# 3. Cálculo de Sesgo y Precisión
valor_real <- mean(bank_data$balance) # Valor real de la población
valor_est <- coef(res_media)[[1]]      # Valor que estimamos con la muestra
error_est <- SE(res_media)[[1]]        # Error estándar

sesgo_abs <- valor_est - valor_real
sesgo_rel <- (sesgo_abs / valor_real) * 100

# 4. Cálculo de Intervalos de Confianza (95%)
ic <- confint(res_media)

# --- REPORTE DE CALIDAD ---
cat("---- RESULTADOS DEL ANÁLISIS ----\n")

```

---- RESULTADOS DEL ANÁLISIS ----

```
cat("Valor Real Poblacional: ", round(valor_real, 2), "\n")
```

Valor Real Poblacional: 1422.66

```
cat("Valor Estimado (Muestra):", round(valor_est, 2), "\n")
```

Valor Estimado (Muestra): 1522.46

```
cat("Sesgo Relativo:          ", round(sesgo_rel, 2), "%\n")
```

Sesgo Relativo: 7.02 %

```
cat("Intervalo de Confianza: [", round(ic[1], 2), ",", round(ic[2], 2), "]\n")
```

Intervalo de Confianza: [1043.2 , 2001.73]

Efecto de Sesgo

```
# Valor Real
valor_real <- mean(bank_data$balance)

# Valor Estimado (del diseño de dos etapas anterior)
valor_est <- coef(res_media) # Extraemos la media del objeto de survey

# Sesgo Absoluto y Relativo
sesgo_abs <- valor_est - valor_real
sesgo_rel <- (sesgo_abs / valor_real) * 100

cat("Sesgo Absoluto:", sesgo_abs, "\n")
```

Sesgo Absoluto: 99.80701

```
cat("Sesgo Relativo:", sesgo_rel, "%\n")
```

Sesgo Relativo: 7.015531 %

Intervalos de Confianza

```
# Intervalo de confianza al 95% para el diseño de dos etapas
intervalo <- confint(res_media, level = 0.95)
print(intervalo)
```

```
      2.5 %    97.5 %
balance 1043.197 2001.732
```

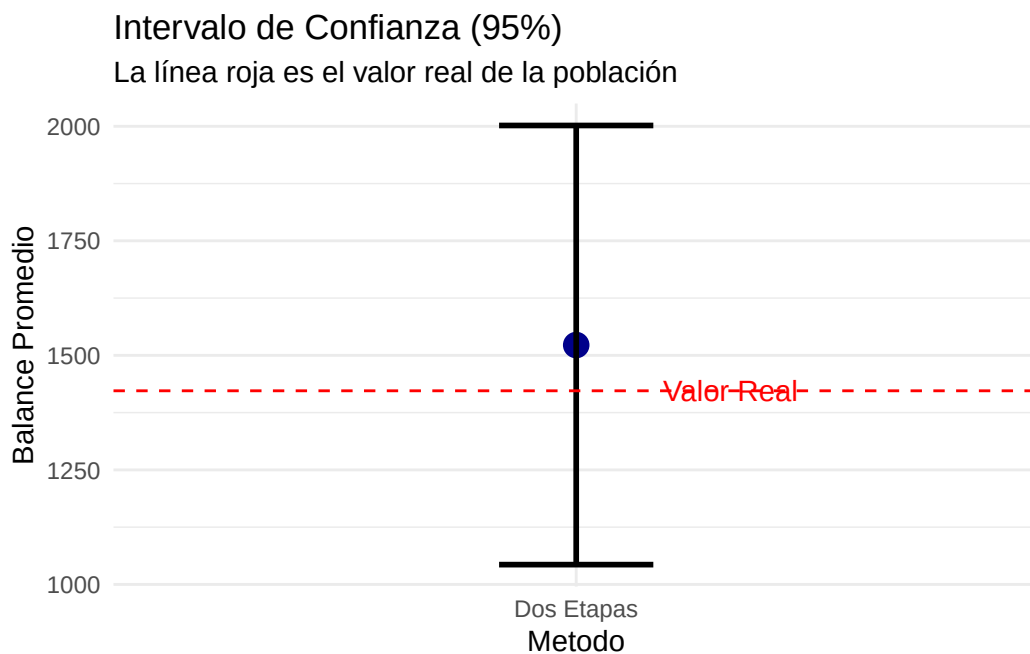
```
# Visualización del IC
res_plot <- data.frame(
  Metodo = "Dos Etapas",
  Media = as.numeric(valor_est),
  Lower = intervalo[1],
```

```

    Upper = intervalo[2]
  )

ggplot(res_plot, aes(x = Metodo, y = Media)) +
  geom_point(size = 4, color = "darkblue") +
  geom_errorbar(aes(ymin = Lower, ymax = Upper), width = 0.2, lwd = 1) +
  geom_hline(yintercept = valor_real, linetype = "dashed", color = "red") +
  annotate("text", x = 1.2, y = valor_real, label = "Valor Real", color = "red") +
  labs(title = "Intervalo de Confianza (95%)",
       subtitle = "La línea roja es el valor real de la población",
       y = "Balance Promedio") +
  theme_minimal()

```



Linearizacion de Taylor

```

library(survey)

# 1. Definimos el diseño (esto activa internamente Taylor para las varianzas)
diseno_complejo <- svydesign(
  id = ~cluster_id,
  weights = ~peso_w,
  data = muestra_2etapas
)

```

```
# 2. Cuando calculamos la media, R usa Taylor para el Error Estándar (SE)
res_media <- svymean(~balance, diseno_complejo)
```

```
# 3. Podemos ver la varianza linealizada explícitamente
vcov(res_media)
```

```
      balance
balance 59794.3
```

```
# Calculamos el CV para ver la estabilidad de la linealización
cv_estimacion <- cv(res_media)
print(paste("El CV es:", round(cv_estimacion * 100, 2), "%"))
```

```
[1] "El CV es: 16.06 %"
```

Estimador de una Razon R

```
library(survey)

# Supongamos que queremos la razón entre 'balance' (saldo) y 'age' (edad)
# Es decir: ¿Cuánto saldo promedio hay por cada año de vida del cliente?

# 1. Usamos el diseño de dos etapas que ya definimos
razon_balance_edad <- svyratio(numerator = ~balance,
                              denominator = ~age,
                              design = diseno_final)

# 2. Ver el resultado y su error estándar
print(razon_balance_edad)
```

```
Ratio estimator: svyratio.survey.design2(numerator = ~balance, denominator = ~age,
      design = diseno_final)
```

Ratios=

```
      age
balance 36.58745
```

SEs=

```
      age
balance 6.080606
```

```
# 3. Obtener el intervalo de confianza de la razón
confint(razon_balance_edad)
```

```
          2.5 %    97.5 %
balance/age 24.66968 48.50522
```

```
# Razón Saldo / Duración de la llamada
razon_campana <- svyratio(~balance, ~duration, diseno_final)
print(razon_campana)
```

```
Ratio estimator: svyratio.survey.design2(~balance, ~duration, diseno_final)
```

```
Ratios=
```

```
      duration
balance 5.701718
```

```
SEs=
```

```
      duration
balance 1.456526
```

Jackknife y Bootstrap

```
# Convertir el diseño a réplicas Jackknife (JK1 o JKn)
diseno_jk <- as.svrepdesign(diseno_final, type = "JK1")
```

```
# Estimar la media usando Jackknife
media_jk <- svymean(~balance, diseno_jk)
print(media_jk)
```

```
      mean    SE
balance 1522.5 245.89
```

```
# Convertir a diseño de réplicas Bootstrap
# R = 500 significa que hará 500 simulaciones
diseno_boot <- as.svrepdesign(diseno_final, type = "bootstrap", replicates = 500)
```

```
# Estimar la media usando Bootstrap
media_boot <- svymean(~balance, diseno_boot)
print(media_boot)
```

```
      mean    SE
balance 1522.5 229.67
```



```
# Comparar Errores Estándar
comparativa_se <- data.frame(
  Metodo = c("Taylor", "Jackknife", "Bootstrap"),
  SE = c(SE(res_media), SE(media_jk), SE(media_boot))
)
print(comparativa_se)
```

	Metodo	SE
1	Taylor	244.5287
2	Jackknife	245.8882
3	Bootstrap	229.6688